

Stock Example

In this example we will consider a sample of 1000 randomly chosen NYSE stocks. For each, a time series of their closing price is obtained, covering 30 trading days from August 18, 2017 to September 29, 2017.

Exercise: What are potential issues that could arise if we calculate pairwise distances between these time series using Euclidean distance?

Imagine observe $X_i = \mu_i + \epsilon_i$, $\epsilon_i \sim N(0, \sigma^2)$
 $Y_i = v_i + \delta_i$, $\delta_i \sim N(0, \sigma^2)$
 $i = 1, 2, \dots, p$

Assume ϵ_i, δ_i are independent

Consider μ_i, v_i are the "important" info.

Euclidean dist. $\underline{X}$ and $\underline{Y}$:

$$\begin{aligned} \sum (X_i - Y_i)^2 &= \sum (\underbrace{X_i - \mu_i}_{\epsilon_i} + \underbrace{\mu_i - v_i}_{\text{diff}} + \underbrace{v_i - Y_i}_{\delta_i})^2 \\ &= \sum (X_i - \mu_i)^2 + \sum (\mu_i - v_i)^2 + \sum (v_i - Y_i)^2 \\ &\quad + \text{cross terms} \end{aligned}$$

Take expected value:

$$\begin{aligned} &= p\sigma^2 + \sum (\mu_i - v_i)^2 + p\sigma^2 \\ &= \underbrace{\text{Euclidean dist. between } \mu_i, v_i}_{\text{actually of interest}} + \underbrace{2p\sigma^2}_{\text{contamination from error}} \end{aligned}$$

Could smooth time series first

The approach we will take here is to transform each time series into a sequence of True/False based on whether or not the price increased or not versus the previous day.

```
In [4]: import pandas as pd
import numpy as np

stocksample = pd.read_csv("stocksample.txt",
                           sep="\t")

stocksample['sector'].fillna("None",
                             inplace=True)

stocksampleupdown = stocksample.iloc[:, 4:34]
stocksampleupdown = np.diff(stocksampleupdown,
                             axis=1) > 0
```

We calculate the distance between a pair by calculating the proportion of the entries in a pair of vectors which disagree. This is referred to as the **Hamming Distance**.

```
In [5]: from scipy.spatial.distance \
import pdist, squareform

stockdistmat = \
    squareform(pdist(stocksampleupdown,
                      metric='hamming'))
```

Now run Isomap, with $K = 5$, and retaining a five-dimensional representation of the objects.

```
In [6]: from sklearn.manifold import Isomap

isooutstocks = Isomap(n_neighbors=5,
                      n_components=5).fit_transform(stockdistmat)
```

Now create a plot looking at the first two dimensions, coloring the points by sector:

```
In [7]: import seaborn as sns

stocksampleiso = pd.concat([stocksample,
                           pd.DataFrame(isooutstocks, columns=\
                                         ["iso1", "iso2", "iso3", "iso4", "iso5"])],
                           axis=1)

sns.set(rc={'figure.figsize': (9, 9)})

ax = sns.scatterplot(x="iso1", y="iso2",
                    data=stocksampleiso, s=30, hue="sector")
plt.legend(bbox_to_anchor=(1.0, 1.0))
plt.xlabel("U[1]")
plt.ylabel("U[2]")
plt.show()
```



Exercise: Execute the commands below and explore the structure.

```
In [9]: import plotly.express as px
```

```
fig = px.scatter_3d(stocksampleiso, x='iso1',
                    y='iso2', z='iso3', hover_name="name",
                    color="sector", opacity=0.7)
fig.show()
```

Diffusion Map

A criticism of Isomap is that it is computationally intensive to calculate the geodesic distances. Also, Isomap is not as robust to noise in the data as are some other approaches, including the one described next, **diffusion map**.

This approach is again built on MDS, but the underlying distance between pairs of points is calculated from **random walks** constructed on the observed data. Steps in this random walk are taken from observation to observation based on proximity in Euclidean distance (typically), but only neighbors at short distances are candidates for stepping to.

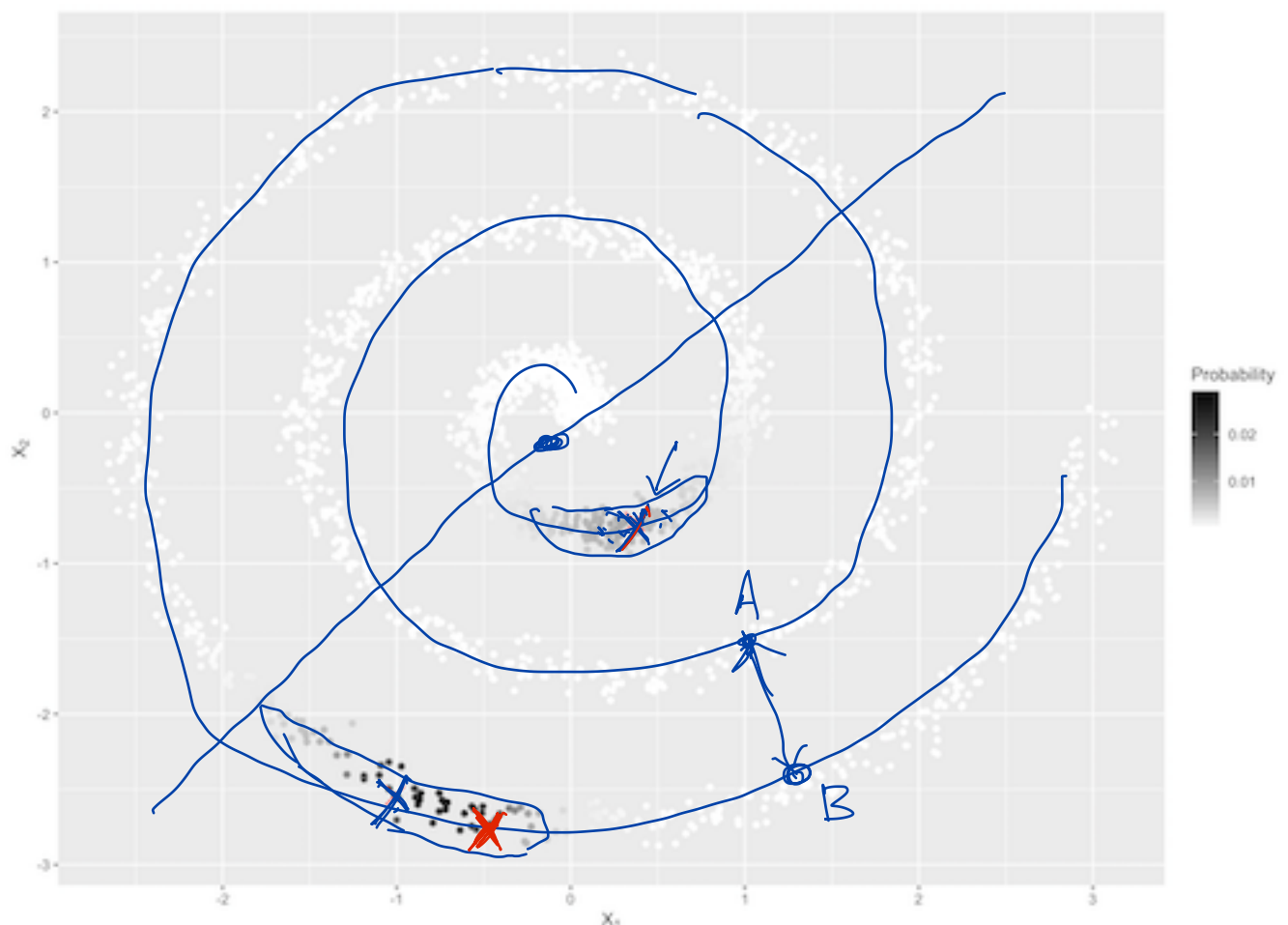
If the random walk “diffuses” over enough steps, the result is a **distribution** over the observations. Comparing distributions between different starting points gives a natural measure of global proximity.

The figure below illustrates the idea.

Two observations in the data set are marked with “X.” For either X, imagine a random walk that takes ten steps away from that observation. Steps are small, i.e., the probability of stepping to an observation decreases rapidly as a function of its distance from the current position in the walk.

The ending position of the random walk is, of course, random. The shading shows the probability that the random walk ends at various observations.

The dissimilarity of these distributions is a natural way of quantifying the distance between them.



Technical Details on Diffusion Map

The process starts by constructing a **similarity measure** between pairs of observations. There is some flexibility in how this is done, but a standard approach is quantify the similarity between observations i and j as

$$s_{ij} = \exp(-\|x_i - x_j\|^2 / \epsilon)$$

where ϵ is a tuning parameter.

Handwritten notes: s_{ij} large when x_i, x_j are "close".
 ↑
 Euclidean Dist.

Next, the probability of stepping from observation i to any other observation j is

$$p_{ij} = s_{ij} / \sum_{k=1}^n s_{ik}$$

Handwritten notes: p (with an arrow pointing to the denominator)

Note that this scaling will force $\sum_j p_{ij} = 1$.

This completely defines the behaviour of the random walk.

The choice of ϵ is crucial. If it is chosen too small, then steps of the random walk will likely move only to other observations which are very similar, and, in particular, remain at the same observation. (A "step" can be to the same observation.)

If ϵ is chosen too large, then it will be possible to step to observations which are actually quite dissimilar, and the result will be a mapping that does not adhere to the lower dimensional structure of interest.

There is no "right" choice of ϵ , but there are algorithms for making (hopefully) informed choices, including that of Berry, Giannakis and Harlim.

Below shows the results when running diffusion map on the S-Curve example.

In this case, ϵ is chosen by the Berry, Giannakis and Harlim algorithm.

The random walk is much more robust to noise in the data than is the geodesic distance. One way to see this is to note that small perturbations in the position of any observation will have very little effect on the diffusion distances, but could potentially affect geodesic distances a great deal.

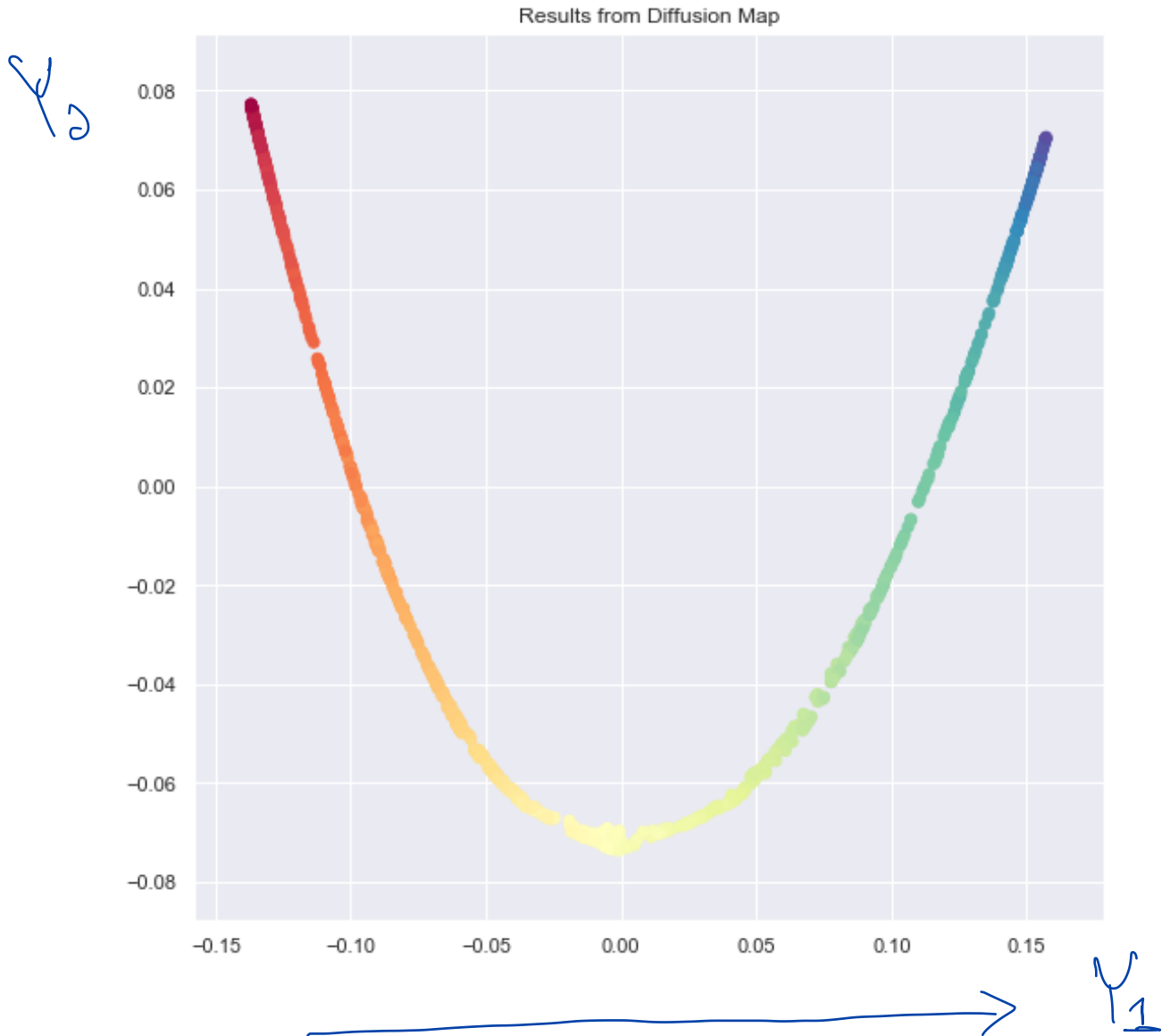
Also, there are computational shortcuts that make calculating the diffusion map quick. Much of this is based on the theory of **Markov chains**.

```
In [10]: from pydiffmap import diffusion_map as dm

        dmap = dm.DiffusionMap.from_sklearn(n_evecs=3,
        epsilon="bgh")

In [11]: Scurvediffmap = dmap.fit_transform(Scurve)

        plt.scatter(Scurvediffmap[:, 0],
        Scurvediffmap[:, 1], c=Scolor,
        cmap=plt.cm.Spectral)
        plt.title("Results from Diffusion Map")
        plt.show()
```

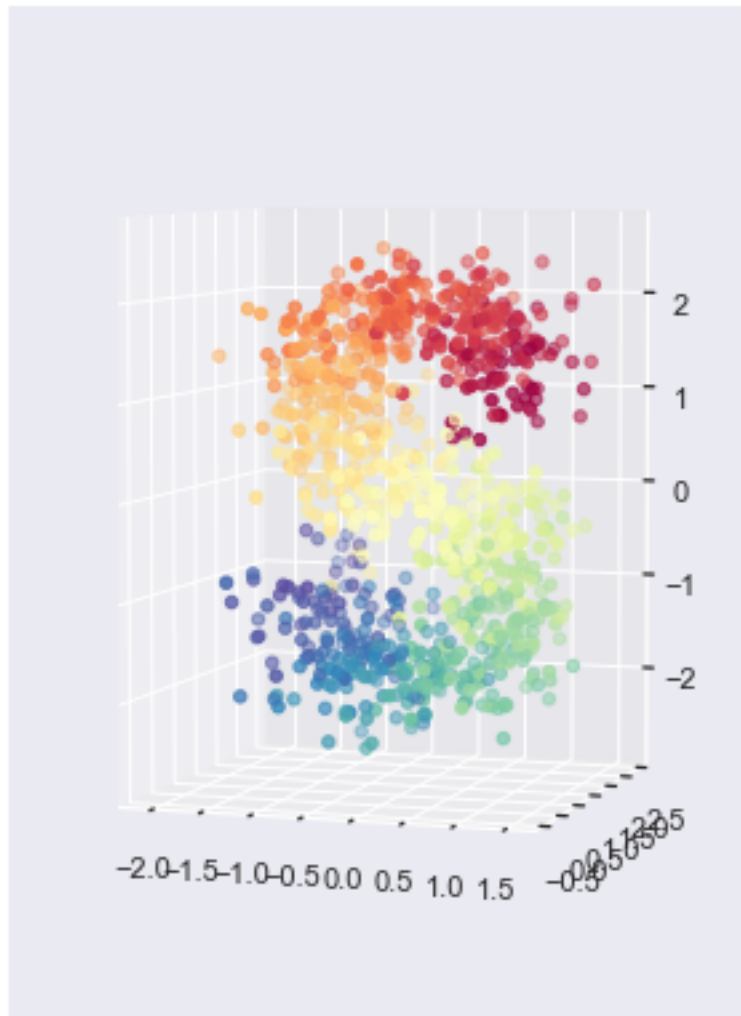


Exercise: Consider the noisy version of the S-curve below. Refit both Isomap and Diffusion Map to this example. Try multiple simulated data sets.

```
In [12]: Snoise = Scurve + \
          np.matrix(np.random.normal(0, .3, 3000)).\
          reshape(1000, 3)

fig = plt.figure(figsize=(5, 7))

ax = fig.add_subplot(111, projection='3d')
ax.scatter(Snoise[:, 0], Snoise[:, 1],
          Snoise[:, 2], c=Scolor,
          cmap=plt.cm.Spectral)
ax.view_init(4, -72)
plt.show()
```



Return to the Stock Example

Let's revisit the stock time series data. This time, we will use the original time series, but rescale each so that they have mean zero and variance one.

```
In [13]: stocksamplescl = stocksample.iloc[:,4:34]

        for i in range(1000):
            stocksamplescl.iloc[i,:] = \
                (stocksamplescl.iloc[i,:] - \
                 np.mean(stocksamplescl.iloc[i,:]))/\
                 np.std(stocksamplescl.iloc[i,:])
```

Run the diffusion map procedure.

```
In [14]: dmap = dm.DiffusionMap.from_sklearn(n_evecs=3,
        epsilon=50)

        dmapoutstocks = \
            dmap.fit_transform(stocksamplescl)
```

Exercise: Create the figure to visualize the results. Comment on what you find.

```
In [15]: stocksampledmap = pd.concat([stocksample,
    pd.DataFrame(dmapoutstocks, columns=\
    ["dmap1", "dmap2", "dmap3"])],
    axis=1)

fig = px.scatter_3d(stocksampledmap, x='dmap1',
    y='dmap2', z='dmap3', hover_name="name",
    color="sector", opacity=0.7)
fig.show()
```

How to find location for new observation?

Nystrom Extension - find distances

from new obs. to everything in
training set